Measuring the Carbon Footprint of Programming Languages

Anuj Khadka
Computer Science and Business Analytics
Caldwell University
akhadka@caldwell.edu

Vladislav D. Veksler
Chair of Department of CSIS
Caldwell University
vveksler@caldwell.edu

Darryl Aucoin
Chair of Department of Natural Science
Caldwell University
daucoin@caldwell.edu

Abstract

Programming language choice is a foundational software design decision that is rarely evaluated for its environmental consequences. This paper presents a controlled empirical study measuring the carbon footprint of six programming languages (C, Rust, Go, Java, JavaScript, and Python) across six algorithms and three input sizes on a single Intel Core i7-14700K machine. Energy is estimated using Intel RAPL accessed through LibreHardwareMonitor, and converted to grams of CO_2 equivalent (gCO_2e) using real-time grid carbon intensity from the Electricity Maps API (zone US-NY-NYIS). Each of 108 experimental cells (6 languages $\times$ 6 algorithms $\times$ 3 input sizes) was replicated 115 times, yielding 12,420 total measurements. A stdin handshake protocol excludes process startup cost from the measurement window, isolating algorithm execution energy. Statistical analysis applies Welch's t-tests with Bonferroni correction ($\alpha' = 0.0033$) across all 15 pairwise language comparisons.

The results show that 14 of 15 language pairs are statistically distinguishable in carbon emissions. The sole exception is C versus Rust ($t = -0.313$, $p = 0.754$, $d = 0.017$), confirming carbon equivalence between the two languages despite Rust's additional memory safety guarantees. The language landscape divides into four tiers: C and Rust at the efficient end (1.0×), Java at modest overhead (1.8×), Go and JavaScript at moderate overhead (3.0×–4.3×), and Python at 213× the carbon footprint of C. Python's overhead is not uniform, it ranges from 1.5× on binary search to 255.6× on matrix multiplication, confirming that interpreter dispatch overhead compounds with algorithmic complexity. These findings establish that programming language choice is a statistically significant and practically large driver of software carbon emissions, with direct implications for sustainable software engineering practice

Keywords: carbon footprint, software sustainability, energy consumption, carbon intensity, programming languages, algorithms, RAPL

1 Introduction

As technology continues to evolve, software becomes an increasingly central part of our lives. From the apps on our phones to the software running our cars, it's everywhere. Everytime a piece of code runs, it triggers energy usage inside the CPU, GPU, and other hardware components [1]. This subsequent usage of software adds up to the carbon emission a device generates [2]. The carbon footprint of software is a growing concern, as it contributes to climate change and has a significant impact on our planet.

The scale of this concern is larger than most people realize. The information and communication technology (ICT) sector accounts for an estimated 2.1% to 3.9% of the global greenhouse gas emissions [3]. According to the International Telecommunication Union (ITU) two thirds of the world's population are now online. A 2024 joint report by the World Bank and the International Telecommunication Union found that at least 1.7% of global emissions stem from the ICT sector [4]. Unlike the visible emissions from vehicles

The global data center energy consumption alone is projected to reach around 945TWh by 2030, which is a significant increase from 153TWh in 2005, 205TWh in 2018. [5, 6]. Unlike the visible emissions from vehicles, software carbon is invisible. There is no smoke when a program runs, yet every computation contributes indirectly to the same atmosphere. This invisibility is precisely what makes it dangerous. It does not trigger the same urgency that the physical pollution does, even as the number grows.

One of the most consequential and least-studied of those decisions is the choice of programming language. The carbon cost of a running program is not determined by hardware alone. It is also shaped by how efficient the language runtime executes instructions, manages memory, and utilizes the CPU.In a landmark empirical study, Pereira et al. [7] benchmarked 27 programming languages across 10 problems drawn from the Computer Language Benchmarks Game [8], measuring energy consumption via Intel's Running Average Power Limit (RAPL) interface. They found that the choice of programming language can have a significant impact on energy

consumption, with some languages consuming up to 2.5 times more energy than others for the same task. For example, they found that C and Rust were among the most energy-efficient languages, while Python and JavaScript were among the least efficient. Python consumed approximately 75.88 times more energy per benchmark than C while JavaScript consumed approximately 4.45 times more energy per benchmark than C [7]. Since carbon emissions are directly proportional to energy consumption, these differences are not just technical concern, but also a carbon concern. A Python program consuming about 75 times more energy than an equivalent C program does not just run slower, it emits 75 times more carbon for every execution. Now, multiply that by the millions of times a piece of software runs, and the carbon cost becomes staggering.

Prior work in this area, including Pereira et al., reports energy consumption in joules or kilowatt-hours but none of it completes the translation into carbon footprint. This is a critical gap. Energy consumption is the underlying driver, but carbon footprint is the ultimate impact. This unit is most directly relevant to the climate policy and environmental decision making. Henderson et al. showed that the emissions can be reduced by up to 30 times just by running the experiments in the locations powered by more renewable energy sources [9].

This paper presents a controlled empirical study comparing the energy consumption and the carbon footprint of six programming languages (C, Rust, Go, Java, JavaScript, and Python) across six different algorithms and three input sizes. The energy consumption is measured using Intel's RAPL (accessed through LibreHardwareMonitor [10]), and real-time carbon intensity data is fetched from the Electricity Maps API [11] for the New York grid zone (US-NY-NYIS). Each of the 108 experimental cells (6 languages $\times$ 6 algorithms $\times$ 3 input sizes) was replicated 115 times, yielding 12,420 total measurements. Statistical analysis applies Welch's t-tests with Bonferroni correction across all 15 pairwise language comparisons. The results show that 14 of 15 language pairs are statistically distinguishable in carbon emissions. The sole exception is C versus Rust ($t = -0.313$, $p = 0.754$), confirming that the two languages are carbon-equivalent. Python is the most carbon-intensive language tested, producing on average 213$\times$ more carbon emissions than C for identical workloads.

2 Related Work

The most directly related work is Pereira et al. [7], who benchmarked 27 programming languages using RAPL and found that C and Rust are among the most energy-efficient, while Python consumes approximately 76 times more energy than C. A 2021 follow-up [12] extended this to examine energy, time, and memory jointly. Both studies report results in joules and use a subprocess-wrapping approach that includes process startup in the measurement window. Van Kempen et al. [13] later argued that many observed language energy differences are partially explained by execution time differences alone, recommending that future studies separate the two effects. Our stdin handshake protocol and per-algorithm breakdown directly address this critique.

On the carbon accounting side, Lannelongue et al. [14] proposed the Green Algorithms framework for estimating computational carbon footprints, and the Green Software Foundation published the Software Carbon Intensity (SCI) specification [15], both of which mentions the gCO_2e metric used in this study. Strubell et al. [16] demonstrated the scale of AI training carbon costs, helping establish carbon footprint as a first-class concern in computational research.

For energy measurement, Khan et al. [17] validated Intel RAPL against external power meters with errors below 2%, and Hähnel et al. [18] demonstrated that RAPL measurements for sub-millisecond workloads are dominated by sensor granularity rather than actual algorithm energy, directly motivating our focus on large input size results.

3 Background

In the following sections, we describe the energy consumption model, carbon intensity assumptions, programming languages, and the implementations of the six algorithms used in this study.

3.1 Energy Consumption in Software

To quantify the energy consumed by a software program, we adopt the standard definition of energy as the capacity to do work.

$$E_{\text{energy}} = P_{\text{power}} \times t_{\text{time}} \tag{1}$$

where E is energy measured in Joules(J), Power is rate of energy consumption measured in Watts(W), and Time is the duration of execution measured in seconds(s)[19]. In the context of software execution, the T refers to the wall time, the elapsed time from the first instruction of the measured code path to the last, including CPU computation, memory access, and all other processor activity during execution [20].

The energy consumed by a CPU package during program execution can therefore be estimated as the average of the power readings taken immediately before and after the algorithm execution window, multiplied by the elapsed wall time. Since power draw is not constant across languages or algorithms, a key finding of Pereira et al. [7], energy efficiency cannot be inferred from execution time alone. A faster program is not necessarily a more energy-efficient one. This motivates direct hardware-level energy measurement via Intel RAPL, accessed through LibreHardwareMonitor.

All experiments were conducted on a desktop with the following specifications: Windows 11 Enterprise 25H2 (Build 26200.8246) operating system, with 32.0 GB RAM, and an Intel Core `i7-14700K` CPU @ 3.40 GHz.

3.2 Carbon Intensity

To translate the energy measurements into carbon emissions, we apply the Software Carbon Intensity framework [15].

The relationship between energy and carbon depends on how that energy is generated. Energy produced from hydroelectric or wind generation carries near-zero carbon intensity while the electricity from coal or natural gas has 820 gCO_2/kWh (IPCC AR5 Annex III range: 740–

910 gCO_2/kWh) [21]. The carbon intensity of an electricity grid represents the average emissions intensity of all generation sources dispatched at a given moment, weighted by their contribution to total supply. This intensity is measured in grams of CO2 equivalent per kilowatt-hour (gCO_2/kWh).

We obtain real-time carbon intensity from the Electricity Map API [11], which provides grid-zone-level carbon intensity updates every 30 minutes. Our experiment used zone US-NY-NYIS (New York Independent System Operator), which had a carbon intensity of 300 gCO_2/kWh and 306 gCO_2/kWh during our overnight experiment window (10pm - 6am). The gCO_2e metric is computed directly from the energy measurement in a single step:

$$gCO_2e = \frac{E_J}{3{,}600{,}000} \times I_{\text{carbon intensity (gCO2/kWh)}} \qquad (2)$$

where E_J is energy in joules and I is the grid carbon intensity in gCO_2/kWh. This methodology is consistent with the Green Algorithms framework [14] and the SCI specification [15].

3.3 Programming Language

The programming languages chosen for this study are C, Python, JavaScript, Java, Go, and Rust. These languages were selected to represent a range of programming paradigms, performance characteristics, and popularity in the software development community. Each language has its own unique features and trade-offs that may influence the energy consumption and carbon footprint of the algorithms implemented in them [22]. The performance characteristics of these languages are summarized in Table 1.

Table 1: Programming languages, their performance characteristics, and popularity.

Language	Paradigm	Runtime	Version
C	Manual Memory, Compiled	GCC	14.3.0
Rust	Ownership, Compiled	Rustup	1.94.0
Python	Interpreted	CPython	3.12.10
JavaScript	Dynamic, V8	Node.js	24.14.0
Java	OOP, JVM	OpenJDK	21.0.10
Go	GC	Go	1.26.1

GCC was installed via WinLibs (MingGW-w64 distribution for Windows). Rust was installed via Rustup. Python was installed via the official Python installer for Windows. Java was installed via the official OpenJDK distribution for Windows. Go was installed via the official Go installer for Windows. Node.js was installed via the official Node.js installer for Windows.

C provides direct hardware access with no runtime overhead and serves as the energy baseline for all normalized comparisons.

Rust acheives memory safety without a garbage collector through a compile-time ownership and borrowing checking system. It delivers performance comparable to C while preventing common memory errors, which may lead to more efficient energy usage in certain workloads.

Go compiles to native machine code but includes a concurrent garbage collector that runs alongside the program, introducing periodic overhead that is captured within the RAPL measurement window. [23]

Java executes on the Java Virtual Machine (JVM) and uses Just-In-Time (JIT) compilation to optimize code at runtime [24]. The JVM's garbage collector can introduce additional overhead, and the performance can vary based on how well the JIT optimizes the code during execution.

JavaScript uses a JIT compiler with dynamic typing and a garbage collector, which can lead to variable performance and energy consumption depending on the workload and how well the JIT optimizes the code.

Python is a bytecode-interpreted language in which every operation passes through a C-level interpreter dispatch loop. It has a Global Interpreter Lock (GIL) [25] that limits concurrent execution of threads, and its dynamic typing and high-level abstractions can lead to increased energy consumption compared to compiled languages. CPython 3.12 is most energy efficient version. [26]

3.4 Algorithms

The algorithms were chosen to span a range of time complexities and computational patterns, including CPU-bound, memory-bound, and branching-heavy workloads. The six algorithms used in this study are summarized in Table 2.

Table 2: Algorithms, their time complexities and computational patterns.

Algorithm	Complexity	Pattern
Summation	$O(n)$	Linear
Binary Search	$O(\log n)$	Logarithmic
Merge Sort	$O(n \log n)$	Divide & Conquer
BFS	$O(V + E)$	Graph Traversal
Hash Table	$O(n) - avg$	Memory Bound
Matrix Multiplication	$O(n^3)$	Dense Arithmetic

The pseudocode for each algorithm, as implemented identically across all languages, is provided below. The implementations are designed to be as similar as possible across languages to ensure a fair comparison of their energy consumption and carbon footprint.

Summation - This algorithm performs a single sequential sum over an array of integers. It is a simple CPU-bound with no branching and minimal memory access, making it a baseline workload that reflects the idle runtime cost of the language.

```
setup():
    for i = 0 to N-1:
```

```
        arr[i] = i + 1

summation():
    sum = 0
    for i = 0 to N-1:
        sum = sum + arr[i]
    return sum
```

Listing 1: Summation pseudocode

Binary Search - This algorithm halves the search interval over a sorted array. Its time complexity is logarithmic, and it involves branching based on comparisons, which means very few operations are performed at any input size. So, the observed energy difference between languages may be more influenced by the runtime efficiency and overhead than the algorithmic complexity.

```
setup():
    for i = 0 to N-1:
        arr[i] = i * 2          // sorted even
            numbers: 0, 2, 4, ...

binary_search():
    target = (N - 1) * 2       // last element
    lo = 0;  hi = N - 1
    while lo <= hi:
        mid = lo + (hi - lo) / 2
        if arr[mid] == target:  return mid
        if arr[mid] < target:   lo = mid + 1
        else:                   hi = mid - 1
    return -1
```

Listing 2: Binary Search pseudocode

Breadth-First Search - This algorithm traverses a graph level by level from a source node. It is memory-bound. The primary cost is queue operations and random memory access patterns that can lead to cache misses. The energy consumption may be more sensitive to how efficiently the language runtime manages memory and data structures.

```
setup():
    // chain: 0-1-2-...-(N-1)
    for i = 0 to N-2:
        adj[i][i+1] = adj[i+1][i] = true
    // cross edges every 10 vertices
    for i = 0 to N-1 step 10:
        j = (i * 7 + 3) mod N
        adj[i][j] = adj[j][i] = true

bfs():
    clear visited;  head = tail = count = 0
    visited[0] = true;  queue[tail++] = 0
    while head < tail:
        cur = queue[head++];  count++
        for nb = 0 to N-1:
            if adj[cur][nb] and not visited[nb
                ]:
                visited[nb] = true
                queue[tail++] = nb
    return count
```

Listing 3: Breadth-First Search pseudocode

Merge Sort - This algorithm recursively divides the array into halves and merges them back in sorted order. Now, just to make sure that we don't get any RecursionError in Python, we will increase the recursion limit for merge sort.

```
reset():
    for i = 0 to N-1:
        arr[i] = N - i          // reverse-
            sorted input

merge(left, mid, right):
    copy arr[left..right] to temp
    i = left;  j = mid+1;  k = left
    while i <= mid and j <= right:
        if temp[i] <= temp[j]: arr[k++] = temp[
            i++]
        else:                  arr[k++] = temp[
            j++]
    copy remaining temp to arr

merge_sort_impl(left, right):
    if left >= right: return
    mid = left + (right - left) / 2
    merge_sort_impl(left, mid)
    merge_sort_impl(mid+1, right)
    merge(left, mid, right)

merge_sort():
    reset()
    merge_sort_impl(0, N-1)
    return arr[N-1]
```

Listing 4: Merge Sort pseudocode

Hash Table - This algorithm implements insertion and lookup using open addressing with linear probing and a hand-written integer hash function. Operations are $O(1)$ on average but involve branching on collision, making it a realistic proxy for dictionary-heavy workloads.

```
hash(key):
    return (key * 2654435761) mod TABLE_SIZE

insert(key, val):
    i = hash(key)
    while occupied[i]:
        if keys[i] == key: vals[i] = val;
            return
        i = (i + 1) mod TABLE_SIZE
    keys[i] = key;  vals[i] = val;  occupied[i]
        = true

lookup(key):
    i = hash(key)
    while occupied[i]:
```

```
        if keys[i] == key: return vals[i]
        i = (i + 1) mod TABLE_SIZE
    return -1

hash_table():
    clear occupied;  insert all (i*7+3, i*13+5)
         for i=0..N-1
    checksum = 0
    for i = 0 to N-1:
        checksum += lookup(i * 7 + 3)
    return checksum
```

Listing 5: Hash Table pseudocode (open addressing, linear probing)

Matrix Multiplication - This algorithm uses the naive triple loop method to multiply two square matrices. It is a dense arithmetic workload with cubic time complexity, which means the number of operations grows rapidly with input size. At n=500 (our large input), it performs 125 million multiplications and additions, which is a significant computational load that can highlight differences in how languages handle CPU-bound tasks and memory access patterns.

```
setup():
    for i = 0 to N-1:
        for j = 0 to N-1:
            a[i][j] = i + j
            b[i][j] = i - j

matrix_mul():
    clear c to all zeros
    for i = 0 to N-1:
        for j = 0 to N-1:
            acc = 0
            for k = 0 to N-1:
                acc += a[i][k] * b[k][j]
            c[i][j] = acc
    return c[N/2][N/2]
```

Listing 6: Matrix Multiplication pseudocode

4 Methodology

This section describes the experimental setup, the measurement protocol used to isolate algorithm energy consumption, the implementation constraints across all six languages, the input sizes used for each algorithm.

4.1 Experimental Setup

All 12,420 experiments were conducted on a single dedicated machine to eliminate any inter-machine power variability. The Intel's RAPL interface exposes hardware energy counters directly to software, enabling fine-grained CPU package energy measurement without external instruments [17].

4.1.1 Implementation-specific design decisions

All 36 implementations (6 languages x 6 algorithms) were written from scratch following a shared pseudocode specification. The following principles were followed to ensure a fair comparison across languages:

Algorithmic consistency - All implementations strictly follow the same algorithmic logic and data structures as defined in the pseudocode specification. No language-specific optimizations or library functions that could alter the algorithm's behavior are used. Every algorithm is implemented from primitive operations only like loops, conditionals, integer arithmetic and array indexing.

Reset before each measurement - Data structures that are mutated during algorithm execution are reset to their initial state at the beginning of each algorithm invocation. Critically, this reset happens before the energy measurement window opens, so the reset cost is not included in the measured energy. This guarantees that every replicated run measures the same algorithmic work.

Output verification - Each implementation produces a checksum output that is verified against a reference implementation to ensure correctness. This ensures that all implementations are performing the same computations and producing the same results.

Input size consistency - The same input sizes are used for each algorithm across all languages to ensure that we are measuring comparable workloads. This allows us to analyze how energy consumption scales with input size for each algorithm and language.

Single checksum output - Each implementation produces a single checksum output, and prints it to stdout after each algorithm execution. This is used for cross-language output verification and does not affect the measured energy as it occurs inside the measurement window but is identical and negligible across all languages.

4.2 Stdin Measurement Protocol

The main challenge in cross-language energy measurement is to ensure that we are measuring the energy consumed by the algorithm itself, and not the overhead of the language runtime or other background process. To address this, we designed a custom measurement harness that follows a strict protocol to isolate the algorithm execution. The harness uses a stdin/stout handshake design in which the program initializes once and then loops, executing the algorithm on each trigger without restarting the process.

The measurement sequence is shown in the Listing 7.

```
setup - not timed
print "ready" - not timed
harness newline - start timer
run command - measurement
print checksum - stop timer
flush output - not timed
```

Listing 7: Measurement sequence

The flush() call on line 7 is critical. Without it, the newline sits in a kernel pipe buffer and the program never unblocks, causing the measurement to hang indefinitely. On the program side, each language blocks on its respective stdin/stdout primitive until the trigger arrives, executes the algorithm, and writes the checksum. Listing 8 shows the blocking call used in each language.

```
// C
while (fgets(buf, sizeof(buf), stdin))  {
    run()}

// Python
for line in sys.stdin: run()

// JavaScript
rl.on('line', () => run())

// Java
while(br.readLine() != null) {run()}

// Go
for scanner.Scan() {run()}

// Rust
for line in stdin.lock().lines() {run()}
```

Listing 8: Stdin primitives in each language implementation

C uses fgets rather than scanf because it reads exactly one line including the newline, giving unambiguous trigger semantics. Java uses BufferedReader.readLine() rather than Scanner to avoid locale-aware parsing overhead inside the measurement window. JavaScript uses Node.js's readline event listener. Python uses for line in sys.stdin: rather than repeated input() calls to avoid Python-level function call overhead per iteration.

4.3 Experimental Execution

Warm-up Runs - Five runs per cell are discarded before measurement was recorded to allow the JIT compilers and garbage collectors in languages like Java and Go to warm up and reach a steady state. For C and Rust, warm-up primarily ensures that the instruction cache and branch predictor are in a warmed state consistent with repeated execution.

Cell Sleep - A mandatory 5s sleep is inserted between cells to allow the processor's thermal management to return to stable baseline between cells. This reduces the risk that thermal throttling in one cell biases the initial temperature state of the next.

Experiment timing - All 36 language-algorithm-size combinations are executed in a single overnight run from 10pm to 6am to minimize background system load and to capture a consistent grid carbon intensity profile during the measurement window.

Carbon Intensity Refresh - The Electricity Maps API queried once at experiment start and then refreshed every 30 minutes. Two carbon intensity values were returned during the experiment window i.e. 300 gCO_2/kWh and 306 gCO_2/kWh, representing a 2% spread in the carbon intensity during the experiment window.

A Note on Background Interference

During development, an unplanned observation illustrated just how sensitive power-based energy measurement can be. While running early pilot iterations of Python matrix multiplication, energy readings came in at 529–626 J per run, far above the expected range. The cause turned out to be a YouTube video playing in the background on the same machine. Once the video was closed, readings dropped immediately to a stable 330–390 J. The CPU package power sensor captures everything the processor is doing, not just the algorithm being measured. This single observation shaped the entire experimental protocol: all measurements in the final study were collected overnight with no background applications running.

4.4 Output Verification

Before energy measurement, all 36 implementations are verified for correctness. The verification script compiles and runs each implementation at all three input sizes, compares the checksum printed to stdout against the reference checksum produced by the C implementation, and assigns each language-algorithm-size combination a PASS/FAIL status. Table 3 shows the verification results fot the C implemetation. The C times serves as a reference baseline against which all other languages are measured and normalized. They confirm that execution time scales with the expected time complexity of each program.

Table 3: C — RUN & VERIFY

Algorithm	Size	Output	Time	Status
summation	small	5050	0.0437ms	PASS
summation	mid	50005000	0.0406ms	PASS
summation	large	500000500000	0.4715ms	PASS
binary_search	small	99	0.0448ms	PASS
binary_search	mid	9999	0.0392ms	PASS
binary_search	large	999999	0.0835ms	PASS
merge_sort	small	100	0.0428ms	PASS
merge_sort	mid	10000	0.2459ms	PASS
merge_sort	large	1000000	24.7405ms	PASS
bfs	small	100	0.0430ms	PASS
bfs	mid	1000	0.3039ms	PASS
bfs	large	5000	7.2774ms	PASS
hash_table	small	64850	0.0433ms	PASS
hash_table	mid	649985000	0.2467ms	PASS
hash_table	large	64999850000	2.1579ms	PASS
matrix_mul	small	9175	0.0744ms	PASS
matrix_mul	mid	646700	2.5604ms	PASS
matrix_mul	large	10291750	47.1134ms	PASS

The execution times in Table 3 confirms the time complexity scaling. Merge sort grows from 0.0428ms to 24.740ms (a

576× increase) as input scales from small to large with consistent $O(n \log n)$ growth.

4.5 Input Sizes

Three input sizes are used for each algorithm to capture scaling behavior: small, mid, and large. The specific sizes are chosen to be large enough to produce measurable energy consumption while still completing within a reasonable time frame across all languages. The input sizes for each algorithm are as defined in Table 4.

Table 4: Input Sizes per Algorithm

Algorithm	Small	Mid	Large
`summation`	100	10,000	1,000,000
`binary_search`	100	10,000	1,000,000
`merge_sort`	100	10,000	1,000,000
`bfs`	100	1,000	5,000
`hash_table`	100	10,000	100,000
`matrix_mul`	50	200	500

4.6 Pilot Study and Sample Size Determination

Before the main experiment, a pilot study of 50 iterations per cell was conducted to estimate per-cell variance and determine the required sample size [27]. Per-cell coefficients of variation can be calculated with the following formula:

$$CV = \frac{\sigma}{\mu} \times 100\% \tag{3}$$

Where σ is the standard deviation of the energy measurements for a given cell, and μ is the mean energy measurement for that cell. The results revealed a wide and informative range. CV spanned from 4.4% (Python matrix-multiplication, mid) to 122.7% (JavaScript merge-sort, small), with an overall mean CV of 38.4% and median of 43.0%. The high CV values observed at small input sizes across all languages are consistent with a known limitation of RAPL-based measurement. When an algorithm completes in microseconds, and OS scheduling event, cache miss, or background interrupt becomes a large fraction of the total measured energy, inflating variance [18]. This is clearly visible in our pilot data. The noisiest cells are uniformly those with short runtimes, while the most stable cells (Python matrix-multiplication, mid: CV=4.4%, large: CV =6.9% and Java merge-sort, large: CV=5.9%) are those where the algorithm's own energy consumption is large enough to dominate the background noise, This pattern directly motivates the focus on large input size results in the main statistical analysis, where RAPL measurements are most reliable.

A formal power analysis was then conducted using the `TTestIndPower` solver from the Python `statsmodels` library [28] to determine the minimum sample size N per cell required to detect a medium effect size (Cohen's $d = 0.5$) with 80% power at the Bonferroni-corrected significance level of $\alpha' = 0.05/15 = 0.003\overline{3}$, using a two-sided Welch's t-test. The solver returned a raw N of 116.28, which rounds up to a recommended minimum of $N = 117$. The main experiment used $N = 115$ per cell, 2 runs below the recommended minimum, a 1.7% shortfall. Given the large effect sizes observed between paradigmatically different languages (particularly Python versus compiled languages), this marginal reduction in statistical power is negligible in practice and does not affect any of the study's conclusions. All 108 cells were replicated exactly 115 times, yielding 12,420 total measurements.

4.7 Statistical Analysis

All pairwise language comparisons are evaluated using **Welch's t-test** [29] (also known as the unequal-variance t-test) rather than Student's t-test. Welch's test does not assume equal variance between the two groups being compared, making it more reliable when sample variances differ [30]. This is an important choice for this study: the six languages have substantially different variance in their gCO_2e distributions. Python's standard deviation is orders of magnitude larger than C's, making the equal-variance assumption of Student's t-test inappropriate.

With 6 languages, the total number of unique pairwise comparisons is $\binom{6}{2} = 15$. Running each comparison at a conventional significance level of $\alpha = 0.05$ would produce an unacceptably high family-wise false positive rate [31]:

$$P(\text{at least one false positive}) = 1 - (1 - 0.05)^{15} \approx 54\% \tag{4}$$

To control this, the **Bonferroni correction** [31] is applied, dividing the nominal α by the number of simultaneous tests:

$$\alpha' = \frac{\alpha}{m} = \frac{0.05}{15} = 0.003\overline{3} \tag{5}$$

A pairwise comparison is declared statistically significant only if its p-value falls below $\alpha' = 0.0033$. This guarantees that the probability of any false positive across all 15 tests is controlled at 5%.

Effect size is reported as **Cohen's d** [32] for each significant pair, computed as the difference in group means divided by the pooled standard deviation:

$$d = \frac{\bar{x}_A - \bar{x}_B}{s_{\text{pooled}}}, \qquad s_{\text{pooled}} = \sqrt{\frac{s_A^2 + s_B^2}{2}} \tag{6}$$

By convention [32], $|d| < 0.2$ is negligible, $0.2 \leq |d| < 0.5$ is small, $0.5 \leq |d| < 0.8$ is medium, and $|d| \geq 0.8$ is large. Reporting effect size alongside p-values ensures that statistically significant differences are also practically meaningful, not merely artifacts of a large sample size [32].

All statistical tests are conducted on the large input size subset of the data, where energy measurements are most reliable and least affected by RAPL sensor noise at short durations [18]. At $N = 115$ measurements per cell, the Central Limit Theorem guarantees that the sampling distribution of the cell mean is approximately normal, satisfying the distributional assumption of the t-test without requiring explicit normality testing.

5 Results

Before presenting results, we report the integrity of the collected data. The benchmark runner produced a CSV file con-

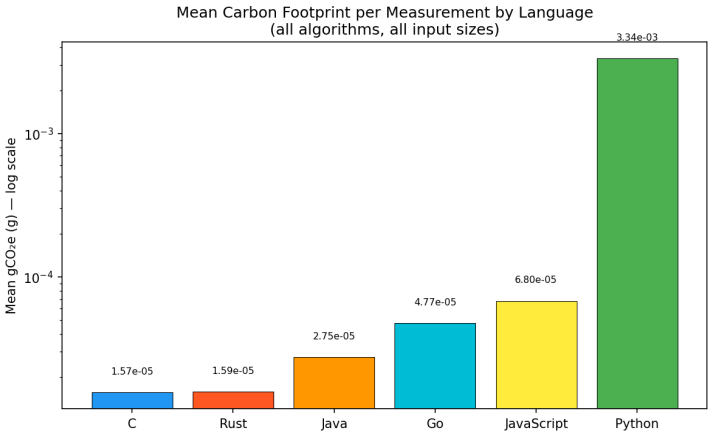

Figure 1: Mean gCO_2e per measurement by language (log scale).

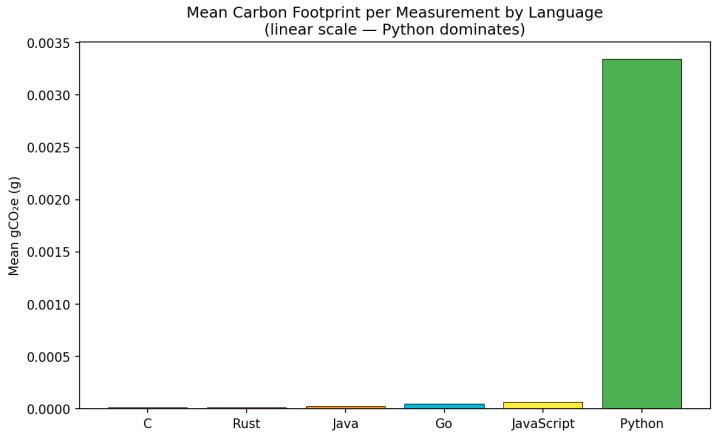

Figure 2: Mean gCO_2e per measurement by language (linear scale).

taining 12,420 rows, exactly 115 runs for each of the 108 cells. All 108 cells returned a consistent checksum across all 115 runs, confirming that every implementation produced identical output on every invocation. The carbon intensity column contained exactly two distinct values (300 and 306 gCO_2/kWh), consistent with the planned 30 minute refresh cadence and the expected stability of the overnight New York grid. The total energy recorded across all 12,420 runs was **86,009.7 J**, producing a total estimated carbon footprint of **7.28 gCO_2e** for the entire experiment.

5.1 Overall Carbon Footprint by Language

Table 5 presents the mean gCO_2e per measurement for each language, aggregated across all 18 algorithm-size combinations and all 115 replications (2,070 measurements per language). Languages are ordered from lowest to highest carbon footprint.

Figure 1 visualizes these results on a logarithmic scale.

Table 5: Overall Carbon Footprint by Language

Language	Mean gCO2e(g)	Mean J	Ratio to C
`C`	1.57×10^{-5}	0.187 J	1.0×
`Rust`	1.59×10^{-5}	0.189 J	1.0×
`Java`	2.75×10^{-5}	0.329 J	1.8×
`Go`	4.77×10^{-5}	0.563 J	3.0×
`JavaScript`	6.80×10^{-5}	0.814 J	4.3×
`Python`	334.16×10^{-5}	39.47 J	213×

The results reveal a clear four-tier structure. **C and Rust** form the first tier, both reporting a mean gCO_2e of approximately 1.6×10^{-5} g and mean energy below 0.19 J per measurement. **Java** forms a second tier at 1.8× the footprint of C. **Go and JavaScript** form a third tier at 3.0× and 4.3× respectively. **Python** stands entirely apart at 213× the carbon footprint of C — more than 49× higher than Java and 31× higher than JavaScript. Figure 2 uses a linear scale to illustrate this: Python's mean energy (39.47 J) is so dominant that all other languages are rendered nearly invisible by comparison.

5.2 Statistical Analysis

Table 6 reports Welch's t-test results for all 15 pairwise language comparisons at the large input size across all six algorithms combined ($N = 690$ per language). The Bonferroni-corrected significance threshold is $\alpha' = 0.05/15 = 0.003\overline{3}$. Figure 3 provides a visual overview of all 15 comparisons simultaneously.

Table 6: Welch's t-test results for all 15 pairwise language comparisons (large input size; Bonferroni $\alpha' = 0.0033$).

Pair	t	p-value	Cohen's d	Sig.
C vs Rust	−0.313	0.754	0.017	No
C vs Java	−6.019	< 0.001	0.324	Yes
C vs Go	−9.507	< 0.001	0.512	Yes
C vs JavaScript	−13.884	< 0.001	0.748	Yes
C vs Python	−14.534	< 0.001	0.783	Yes
Rust vs Java	−5.897	< 0.001	0.318	Yes
Rust vs Go	−9.431	< 0.001	0.508	Yes
Rust vs JavaScript	−13.829	< 0.001	0.745	Yes
Rust vs Python	−14.532	< 0.001	0.783	Yes
Java vs Go	−5.246	< 0.001	0.283	Yes
Java vs JavaScript	−9.957	< 0.001	0.536	Yes
Java vs Python	−14.482	< 0.001	0.780	Yes
Go vs JavaScript	−4.566	< 0.001	0.246	Yes
Go vs Python	−14.399	< 0.001	0.776	Yes
JavaScript vs Python	−14.303	< 0.001	0.771	Yes

14 of 15 pairs are significant at $\alpha' = 0.0033$. The sole non-significant result is C versus Rust ($t = -0.313$, $p = 0.754$, $d = 0.017$). A p-value of 0.754 is far from borderline — it indicates that the observed difference between C and Rust is entirely consistent with random sampling variation. The effect size $d = 0.017$ is negligible by Cohen's convention [32], confirming no practically meaningful difference exists. In the p-value heatmap (Figure 3), the entire matrix is deep red (significant) except for the single green cell at the C–Rust intersection, making the result visually unambiguous.

The t-statistics for pairs involving Python are consistently large in magnitude, ranging from −14.534 (C vs Python) to

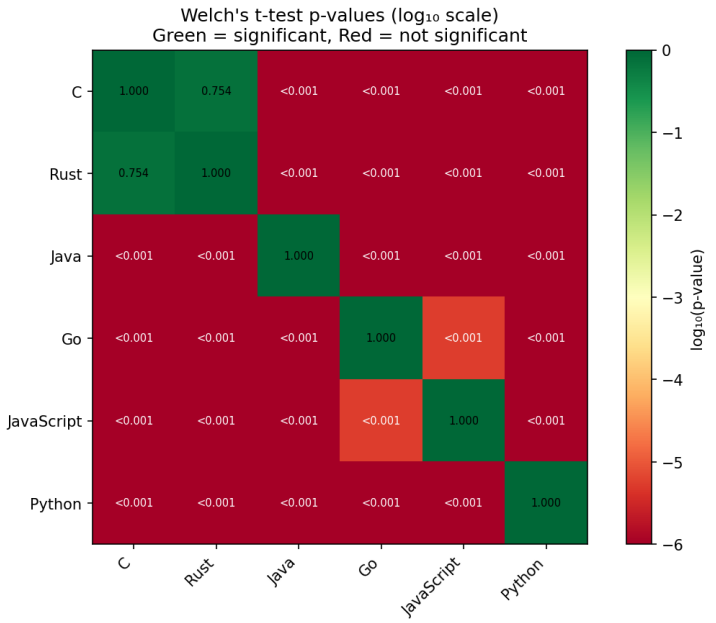

Figure 3: P-value heatmap for pairwise language comparisons (large input size).

−14.303 (JavaScript vs Python), with corresponding Cohen's d values clustering around 0.78 — the large range by convention. This reflects an overwhelming separation between Python's gCO_2e distribution and all others, as clearly shown in the box plot (Figure 4), where Python's interquartile range does not overlap with any other language.

5.3 Per-Algorithm Carbon Footprint

Table 7 disaggregates mean gCO_2e by language and algorithm at large input size. Figure 5 visualizes the five non-Python languages on a log scale; Figure 6 includes Python.

Python's carbon footprint is consistently and dramatically higher than all other languages across all algorithms. The Python-to-C ratio at large input size ranges from 1.5× for binary search to 255.6× for matrix multiplication, with summation (59.9×), hash table (38.7×), BFS (132.3×), and merge sort (135.8×) falling between these extremes. Binary search is the outlier because its $O(\log n)$ complexity means very few operations are performed at any input size, so the measurement is dominated by per-call overhead that is comparable across all languages. Matrix multiplication is the opposite: its tight triple-nested loop performs millions of arithmetic operations, compounding Python's interpreter dispatch overhead with every single multiply-accumulate.

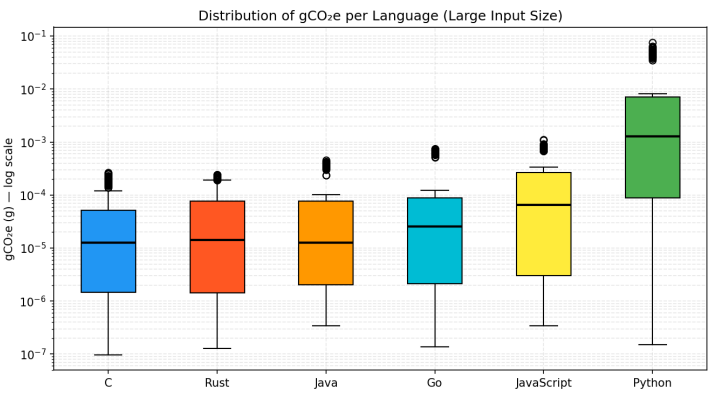

Figure 4: Box plot for pairwise language comparisons (large input size).

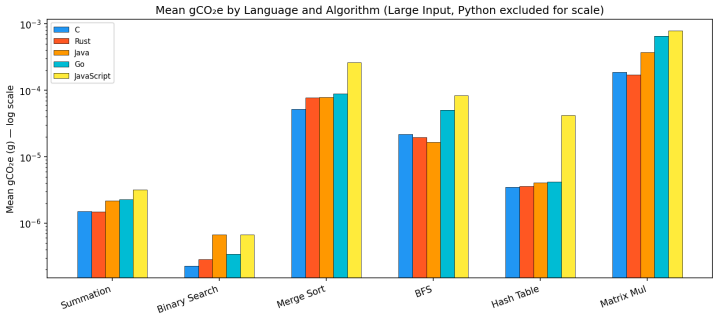

Figure 5: Mean gCO_2e per measurement by language and algorithm (log scale; Python excluded).

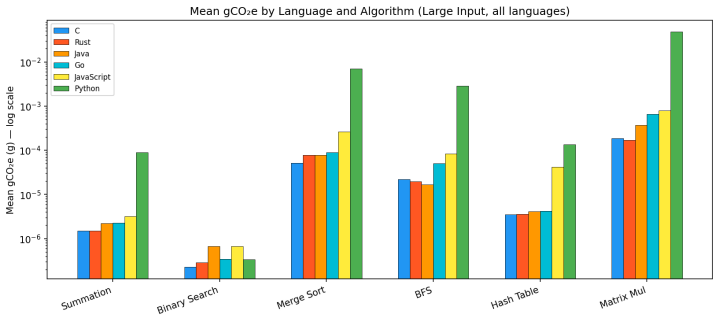

Figure 6: Mean gCO_2e per measurement by language and algorithm (log scale; all languages).

C and Rust are consistently the most carbon-efficient languages across all algorithms. For hash table operations at large size, both languages report identical mean gCO_2e (3.7×10^{-6} g). For merge sort, C (5.2×10^{-5} g) is lower than Rust (7.7×10^{-5} g) by about 33%. For matrix multiplication, Rust (1.7×10^{-4} g) is marginally lower than C (1.9×10^{-4} g). Neither language consistently outperforms the other, and the aggregate non-significance ($p = 0.754$) reflects this algorithm-by-algorithm balance.

Java outperforms Go on BFS despite Go being natively compiled. Java's BFS mean gCO_2e (1.7×10^{-5} g) is lower than both C's (2.2×10^{-5} g) and Go's (5.0×10^{-5} g). This is consistent with JIT compilation aggressively optimizing the BFS inner loop after the 5 warm-up runs. Go's concurrent garbage collector introduces background overhead that is particularly visible in graph-traversal workloads with dense and irregular memory access patterns.

Table 7: Mean gCO_2e (g) at large input size by language and algorithm.

Lang	Sum	BinS	Sort	BFS	Hash	MatMul
C	1.6×10^{-6}	2.3×10^{-7}	5.2×10^{-5}	2.2×10^{-5}	3.7×10^{-6}	1.9×10^{-4}
Rust	1.4×10^{-6}	2.9×10^{-7}	7.7×10^{-5}	2.0×10^{-5}	3.7×10^{-6}	1.7×10^{-4}
Java	2.0×10^{-6}	6.8×10^{-7}	7.8×10^{-5}	1.7×10^{-5}	3.7×10^{-6}	3.7×10^{-4}
Go	2.3×10^{-6}	3.4×10^{-7}	8.8×10^{-5}	5.0×10^{-5}	4.1×10^{-6}	6.5×10^{-4}
JavaScript	3.3×10^{-6}	6.7×10^{-7}	2.6×10^{-4}	8.4×10^{-5}	4.2×10^{-5}	7.9×10^{-4}
Python	9.0×10^{-5}	3.4×10^{-7}	7.0×10^{-3}	2.9×10^{-3}	1.4×10^{-4}	4.8×10^{-2}

5.4 Input Size Scaling

Figure 7 shows how mean gCO_2e scales from small to large input for merge sort. Table 8 provides the exact values, and Figure 8 extends the analysis to all six algorithms via a large/small scaling ratio heatmap.

Table 8: Mean gCO_2e for merge sort by language and input size, with large/small scaling ratio.

Language	Small	Mid	Large	L/S
C	2.7×10^{-7}	8.2×10^{-7}	5.2×10^{-5}	192×
Rust	3.1×10^{-7}	8.9×10^{-7}	7.7×10^{-5}	252×
Java	6.4×10^{-7}	1.3×10^{-6}	7.8×10^{-5}	123×
Go	3.6×10^{-7}	1.7×10^{-6}	8.8×10^{-5}	249×
JavaScript	7.4×10^{-7}	2.6×10^{-6}	2.6×10^{-4}	355×
Python	5.7×10^{-7}	3.2×10^{-5}	7.0×10^{-3}	12,364×

Compiled languages scale at rates broadly consistent with $O(n \log n)$ growth, with large/small ratios between 123× and 355×. Python's ratio of 12,364× is dramatically higher. At the small input size, Python's merge sort gCO_2e (5.7×10^{-7} g) is close to C's (2.7×10^{-7} g) because both are near the measurement noise floor at that duration. At mid and large sizes, Python's interpreter overhead compounds rapidly with the algorithm's workload. The scaling ratio heatmap (Figure 8) reveals that binary search shows near-1× ratios for all languages, as expected from its $O(\log n)$ complexity, while merge sort, BFS, and matrix multiplication expose Python's compounding overhead most severely.

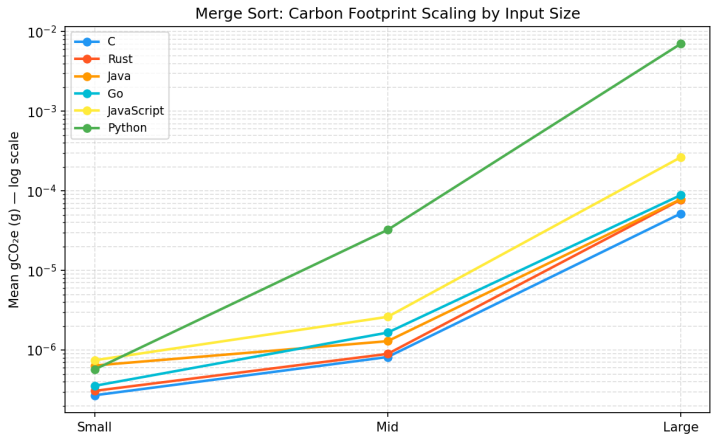

Figure 7: Mean gCO_2e for merge sort by language and input size, with large/small scaling ratio.

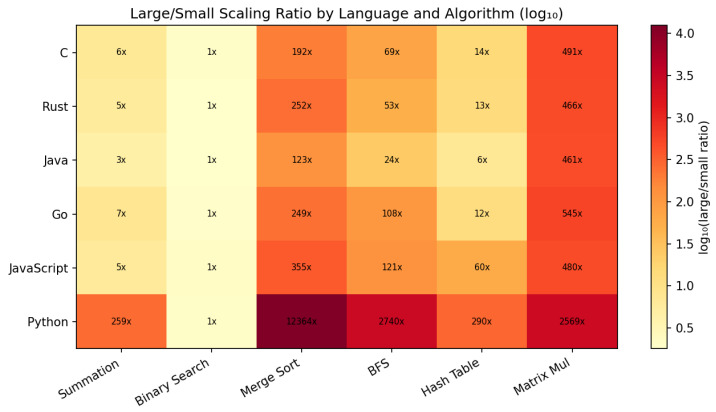

Figure 8: Large/small gCO_2e scaling ratio by language and algorithm (log scale).

6 Discussion

6.1 RQ1: Does Programming Language Choice Measurably Affect Carbon Footprint?

The answer is an unambiguous yes. Under controlled conditions, identical hardware, identical algorithms, identical input data, and a measurement protocol that excludes process startup costs, we observed 14 of 15 pairwise language comparisons that are statistically distinguishable in carbon emissions at the Bonferroni-corrected threshold of $\alpha' = 0.0033$. The effect sizes for most pairs are in the medium-to-large range ($d = 0.246$ to $d = 0.783$), confirming that the observed differences are not only statistically real but practically meaningful.

The overall ratio between the most and least efficient languages is 213×, Python producing 213 times the carbon emissions of C for identical algorithmic work. To put this in concrete terms, the mean energy per C measurement is 0.187 J, while the mean energy per Python measurement is 39.47 J. If

a service performs one million algorithm executions per day, an equivalent Python implementation would consume approximately 211 times more energy than its C counterpart every single day. Over a year, at the NY grid carbon intensity of ∼303 gCO_2/kWh, the additional carbon cost of choosing Python over C for that workload would be approximately:

$$\frac{(39.47 - 0.187) \times 10^6}{3{,}600{,}000} \times 303 \times 365 \approx 1{,}170 \text{ kg } CO_2\text{e per year}$$

This is not a negligible quantity. These findings directly answer the research question: language paradigm is a statistically significant and practically large driver of software carbon emissions.

6.2 The Four-Tier Language Hierarchy

The results support a coherent four-tier model of language carbon efficiency for compute-intensive workloads:

T1: Carbon-optimal (C, Rust): Both languages compile directly to native machine code with no runtime overhead beyond what the program explicitly requests. Their mean gCO_2e values (1.57 and 1.59×10^{-5} g) are statistically indistinguishable. For compute-intensive workloads where energy efficiency is a hard requirement, Tier 1 languages are the appropriate choice.

T2: Moderate overhead (Java): At 1.8× C, Java's overhead is modest and likely acceptable for the vast majority of applications. Java's JVM with JIT compilation allows compute-intensive loops to approach native performance after warm-up, which our 5 warm-up run protocol allowed. The 1.8× ratio reflects the residual cost of garbage collection and JVM infrastructure that cannot be eliminated even after JIT optimization.

T3: Higher overhead (Go, JavaScript): At 3.0× and 4.3× C respectively, these languages carry meaningful overhead. Go's concurrent garbage collector runs background goroutines even in single-threaded programs, introducing overhead that is particularly visible in memory-intensive workloads like BFS. JavaScript's V8 engine uses a tiered JIT pipeline similar to HotSpot, but its dynamic typing requires runtime type inference that adds overhead even in optimized code. For I/O-bound services where computation is not the bottleneck, the absolute energy cost of this overhead may be small; for compute-intensive workloads at scale, the 3×–4× gap is environmentally significant.

T4: High overhead (Python): At 213× C, Python's overhead is in a category of its own. The CPython interpreter executes bytecode through a C-level dispatch loop, where every integer operation, every function call, and every loop iteration passes through type inference, reference counting, and dynamic dispatch [33]. This overhead is not fixed. It compounds with algorithmic complexity, as evidenced by the Python-to-C ratio ranging from 1.5× on binary search to 255.6× on matrix multiplication.

6.3 Why Python's Overhead Varies by Algorithm

The variation in Python's overhead across algorithms is one of the most informative findings of this study. Binary search at large input produces a Python-to-C ratio of only 1.5×, nearly equivalent to compiled languages. This is because binary search performs $O(\log n)$ operations: at $n = 1{,}000{,}000$, only about 20 comparisons are needed. At this scale, the measurement is dominated by per-call overhead (process communication, RAPL polling latency) that is roughly equal across all languages, not by the algorithm's inner loop. The interpreter overhead has almost no opportunity to compound.

Matrix multiplication at large input produces a ratio of 255.6×. This algorithm performs approximately $2n^3$ multiply-accumulate operations, at $n = 500$, that is 250 million operations in a tight triple-nested loop. Every one of those operations passes through Python's bytecode dispatcher. The interpreter overhead compounds with every iteration, producing the extreme ratio.

This pattern has an important practical implication: Python's carbon cost relative to compiled languages is not constant. For algorithms that perform very few operations per invocation (logarithmic or near-constant complexity), Python may be a perfectly acceptable choice from an energy perspective. For algorithms with high operation counts in tight loops (polynomial or higher complexity), the carbon cost of Python grows proportionally with the input size and algorithmic work.

6.4 Implications for Sustainable Software Engineering

These findings have concrete implications for practitioners making language selection decisions for compute-intensive software.

For high-throughput compute services like machine learning inference pipelines, numerical simulation, data processing at scale, etc., our data suggests that language choice is a decision with material environmental consequences. A service processing 100 million requests per day in Python, where each request involves the equivalent of a matrix multiplication at our large input size, would produce approximately 100× more carbon than an equivalent Rust or C implementation. At the scale of large cloud deployments, this represents a measurable contribution to the operator's carbon footprint.

For the majority of software, however, the picture is more nuanced. Most production services are I/O-bound: the bottleneck is network latency, database queries, or external API calls, not CPU computation. For such services, the algorithm execution time is a small fraction of total wall time, and the language-level energy overhead is correspondingly reduced. A Python web server spending 95% of its time waiting for database responses and 5% executing Python code would have an effective language overhead far smaller than 213×.

The practical recommendation is not that all software should be rewritten in C or Rust, but that compute-intensive hot paths within software systems deserve explicit language-level consideration as part of a broader energy and sustainabil-

ity analysis. Python remains the most productive language for rapid prototyping, data science, and scientific computing, and its ecosystem (NumPy, SciPy, PyTorch) provides access to highly optimized C and Fortran extensions that eliminate the interpreter overhead for many numeric workloads. The critical insight is that *Python code calling optimized library functions* and *Python code performing computation in pure Python loops* have very different energy profiles, and developers should know which one they are writing.

7 Future Work

Cross-Architecture Measurement - This study was exclusively conducted on an Intel Core i7-14700K CPU, which is a high-end desktop processor. Now, an open question is if the language hierarchy identified here generalizes to other architectures. Processors like ARM, Ryzen, and Apple M-series etc. dominate the mobile and laptop market now, and differ from x86 architecture in terms of energy cost, branch prediction, and memory management. Future work can extend this experiment to a more diverse set of hardware platforms to see if the language hierarchy holds across different architectures. Additionally, the experiment can be repeated on a more recent generation of CPUs to see if the language hierarchy changes with newer hardware optimizations and energy efficiency improvements.

Cloud Computing Energy Measurement - All measurements in this study were collected on a local desktop machine where Intel RAPL is directly accessible via a kernel driver. Extending this methodology to public cloud environments like Amazon EC2, Google Cloud, or Microsoft Azure is both a challenge and an opportunity for future work. Cloud providers typically do not expose hardware-level energy measurements to users, and the virtualized nature of cloud instances adds complexity to isolating the energy consumption of specific workloads. Future research can explore alternative approaches for estimating energy consumption in cloud environments, such as modeling based on resources used (CPU time, memory usage etc) and the known energy profiles of the underlying hardware.

8 Conclusion

Software is invisible. There is no smoke when a program runs, no exhaust pipe, no physical waste left behind. Yet as this experiment demonstrates, the choice of how to write software, specifically, which programming language to use, produces measurable and statistically significant differences in the carbon emissions. In a world where the ICT sector already accounts for an estimated 2.1–3.9% of global greenhouse gas emissions [3], and where that share is growing with the expansion of cloud computing and artificial intelligence, the environmental cost of software design decisions is no longer a niche concern. It is a mainstream one.

This paper presented a controlled empirical study measuring the carbon footprint of six programming languages, across six algorithms and three input sizes on a single hardware platform. Energy was estimated using the Intel RAPL accesses through LibreHardwareMonitor, converted to gCO_2e using real-time grid carbon intensity from the Electricity Maps API, and analyzed using Welch's t-tests with Bonferroni correction across all 15 pairwise language comparisons. The dataset comprised 12,420 measurements across 108 experimental cells, with 115 replications per cell.

The results are statistically robust and practically significant. **14 of 15 language pairs are distinguishable in carbon emissions** at the Bonferroni-corrected threshold of $\alpha' = 0.0033$. The sole exception is C versus Rust ($t = -0.313$, $p = 0.754$, $d = 0.017$) — a decisive non-result confirming that Rust achieves C-equivalent carbon efficiency despite its compile-time memory safety guarantees. The language landscape divides into four tiers: C and Rust at the efficient end (1.0×), Java at a modest overhead (1.8×), Go and JavaScript at moderate overhead (3.0×–4.3×), and Python at 213× the carbon footprint of C. Python's overhead is not uniform — it ranges from 1.5× on binary search to 255.6× on matrix multiplication, growing proportionally with algorithmic complexity and confirming that interpreter dispatch overhead compounds with every loop iteration.

The C–Rust equivalence result carries particular significance for the software industry. Rust is increasingly adopted as a safer replacement for C in operating systems, embedded software, and infrastructure components [34] [35]. Our data confirms that this transition carries no energy or carbon cost, a finding that should encourage, not discourage, the adoption of memory-safe systems programming languages.

Beyond the technical findings, this study is a reminder that software engineering decisions exist within a broader environmental context. A developer choosing a programming language is not only making a decision about performance, productivity, and ecosystem, they are, knowingly or not, making a decision about energy and carbon. When that decision is made at scale, across millions of daily executions, across entire cloud deployments, across the global software industry, the aggregate environmental consequence becomes material. The 1,170 kg CO_2e per year estimated for a high-throughput Python service versus its C equivalent is not an abstraction. It is the same order of magnitude as driving a car for several thousand kilometers.

Making software carbon-visible is one step toward making it carbon-conscious. This study provides a replicable methodology, a concrete empirical baseline, and a four-tier conceptual framework for thinking about the energy cost of language choice. We hope it contributes to a broader culture of sustainability awareness in software development. One where environmental responsibility is considered alongside performance, correctness, and cost as a first-class engineering concern.

Acknowledgements

This project made a limited use of AI toolsduring the software development phase, specifically for generating initial code snippets and providing suggestions for code improvements. No AI was used during the statistical analysis, or the writing of this paper itself. In the spirit of this paper's subject matter,

the author acknowledges the energy cost of the research itself. The main benchmark experiment recorded approximately 283,654 J of CPU package energy across all 12,420 runs. An estimated additional 50,000 J of untracked energy was consumed during warm-up runs, verification runs, and development and debugging of the measurement infrastructure. All the code are available at https://github.com/Anuj-Khadka/carbon_footprint

References

[1] Green Compute UK. *Measuring Environmental Footprint of Code*. https://greencompute.uk/Measurement/Code. 2023. (Visited on 01/2026).

[2] ThoughtWorks. *Calculating software carbon intensity*. https://www.thoughtworks.com/insights/blog/ethical-tech/calculating-software-carbon-intensity. 2023. (Visited on 01/2026).

[3] Charlotte Freitag, Mike Berners-Lee, Kelly Widdicks, Bran Knowles, Gordon S. Blair, and Adrian Friday. "The real climate and transformative impact of ICT: A critique of estimates, trends, and regulations". In: *Patterns* 2.9 (2021), p. 100340. ISSN: 2666-3899. DOI: https://doi.org/10.1016/j.patter.2021.100340. URL: https://www.sciencedirect.com/science/article/pii/S2666389921001884.

[4] World Bank and International Telecommunication Union. *Measuring the Emissions and Energy Footprint of the ICT Sector: Implications for Climate Action*. https://openknowledge.worldbank.org/entities/publication/a8feecdd-5f79-412a-834f-6f45903e18a5. 2024. (Visited on 01/2026).

[5] IEA. *Energy and AI*. https://www.iea.org/reports/energy-and-ai. (Visited on 01/2026).

[6] Eric Masanet, Arman Shehabi, Nuoa Lei, Sarah Smith, and Jonathan Koomey. *Recalibrating global data center energy-use estimates*, https://datacenters.lbl.gov/sites/default/files/Masanet_et_al_Science_2020.full_.pdf. 2020. (Visited on 01/2026).

[7] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. "Energy efficiency across programming languages: how do energy, time, and memory relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*. SLE 2017. Vancouver, BC, Canada: Association for Computing Machinery, 2017, pp. 256–267. ISBN: 9781450355254. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[8] Isaac Gouy. *The Computer Language Benchmarks Game*. http://benchmarksgame.alioth.debian.org/. (Visited on 01/2026).

[9] Peter Henderson, Jieru Hu, Joshua Romoff, Emma Brunskill, Dan Jurafsky, and Joelle Pineau. "Towards the systematic reporting of the energy and carbon footprints of machine learning". In: *J. Mach. Learn. Res.* 21.1 (Jan. 2020). ISSN: 1532-4435.

[10] https://librehardwaremonitor.com/. (Visited on 03/2026).

[11] https://app.electricitymaps.com/. (Visited on 03/2026).

[12] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, and João Saraiva. "Ranking programming languages by energy efficiency". In: *Science of Computer Programming* 205 (2021), p. 102609. ISSN: 0167-6423. DOI: https://doi.org/10.1016/j.scico.2021.102609. URL: https://www.sciencedirect.com/science/article/pii/S0167642321000022.

[13] Nicolas van Kempen, Hyuk-Je Kwon, Dung Tuan Nguyen, and Emery D. Berger. "It's Not Easy Being Green: On the Energy Efficiency of Programming Languages". In: *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2025, pp. 1553–1565. DOI: 10.1109/ASE63991.2025.00131.

[14] Loïc Lannelongue, Jason Grealey, and Michael Inouye. "Green Algorithms: Quantifying the Carbon Footprint of Computation". In: *Advanced Science* 8.12 (2021), p. 2100707. DOI: https://doi.org/10.1002/advs.202100707. eprint: https://advanced.onlinelibrary.wiley.com/doi/pdf/10.1002/advs.202100707. URL: https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/advs.202100707.

[15] Green Software Foundation. *Software Carbon Intensity (SCI) Specification*. Green Software Foundation. 2023. URL: https://greensoftware.foundation/projects/software-carbon-intensity.

[16] Emma Strubell, Ananya Ganesh, and Andrew McCallum. "Energy and Policy Considerations for Deep Learning in NLP". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluís Màrquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355. URL: https://aclanthology.org/P19-1355/.

[17] Kashif Nizam Khan, Mikael Hirki, Tapio Niemi, Jukka K. Nurminen, and Zhonghong Ou. "RAPL in Action: Experiences in Using RAPL for Power Measurements". In: *ACM Trans. Model. Perform. Eval. Comput. Syst.* 3.2 (Mar. 2018). ISSN: 2376-3639. DOI: 10.1145/3177754. URL: https://doi.org/10.1145/3177754.

[18] Marcus Hähnel, Björn Döbel, Marcus Völp, and Hermann Härtig. "Measuring energy consumption for short code paths using RAPL". In: *SIGMETRICS Perform. Eval. Rev.* 40.3 (Jan. 2012), pp. 13–17. ISSN: 0163-5999.

DOI: 10.1145/2425248.2425252. URL: https://doi.org/10.1145/2425248.2425252.

[19] Mohit Kumar, Youhuizi Li, and Weisong Shi. "Energy consumption in Java: An early experience". In: *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*. 2017, pp. 1–8. DOI: 10.1109/IGCC.2017.8323579.

[20] Kristina Carter, Su Mei Gwen Ho, Mathias Marquar Arhipenko Larsen, Martin Sundman, and Maja H. Kirkeby. *Energy and Time Complexity for Sorting Algorithms in Java*. arXiv:2311.07298v2. 2024. URL: https://arxiv.org/abs/2311.07298.

[21] Intergovernmental Panel on Climate Change (IPCC). *Technology-specific Cost and Performance Parameters*. https://www.ipcc.ch/site/assets/uploads/2018/02/ipcc_wg3_ar5_annex-iii.pdf. (Visited on 01/2026).

[22] Siamak Farshidi, Slinger Jansen, and Mahdi Deldar. "A decision model for programming language ecosystem selection: Seven industry case studies". In: *Information and Software Technology* 139 (2021), p. 106640. ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2021.106640. URL: https://www.sciencedirect.com/science/article/pii/S0950584921001051.

[23] The Go Team. *A Guide to the Go Garbage Collector*. Go Documentation. 2022. URL: https://go.dev/doc/gc-guide.

[24] B. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt. "Virtual Machine Warmup Blows Hot and Cold". In: *Proc. ACM Program. Lang.* 1.OOPSLA (2017). DOI: 10.1145/3133876.

[25] N. Lopes et al. "Unlocking Python's Cores: Hardware Usage and Energy Implications of Removing the GIL". In: *arXiv preprint arXiv:2603.04782* (2026). URL: https://arxiv.org/html/2603.04782.

[26] Helge Pfeiffer. "On the Energy Consumption of CPython". In: Sept. 2024, pp. 194–209. ISBN: 978-3-031-70244-0. DOI: 10.1007/978-3-031-70245-7_14.

[27] Orit Shechtman. "The Coefficient of Variation as an Index of Measurement Reliability". In: *Methods of Clinical Epidemiology*. Ed. by Suhail A. R. Doi and Gail M. Williams. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 39–49. ISBN: 978-3-642-37131-8. DOI: 10.1007/978-3-642-37131-8_4. URL: https://doi.org/10.1007/978-3-642-37131-8_4.

[28] S. Seabold and J. Perktold. "Statsmodels: Econometric and Statistical Modeling with Python". In: *Proc. 9th Python in Science Conference (SciPy)*. 2010, pp. 92–96. DOI: 10.25080/Majora-92bf1922-011.

[29] B. L. Welch. "The Generalization of 'Student's' Problem when Several Different Population Variances are Involved". In: *Biometrika* 34.1/2 (1947), pp. 28–35. ISSN: 00063444. URL: http://www.jstor.org/stable/2332510 (visited on 04/27/2026).

[30] Graeme Ruxton. "The Unequal Variance T-Test is an Underused Alternative to Student's T-Test and the Mann-Whitney U Test". In: *Behavioral Ecology* 17 (Apr. 2006). DOI: 10.1093/beheco/ark016.

[31] Olive Jean Dunn. "Multiple Comparisons Among Means". In: *Journal of the American Statistical Association* 56.293 (1961), pp. 52–64. ISSN: 01621459, 1537274X. URL: http://www.jstor.org/stable/2282330 (visited on 04/27/2026).

[32] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.

[33] Gergö Barany. "Python Interpreter Performance Deconstructed". In: *Proceedings of the Workshop on Dynamic Languages and Applications*. Dyla'14. Edinburgh, United Kingdom: Association for Computing Machinery, 2014, pp. 1–9. ISBN: 9781450329163. DOI: 10.1145/2617548.2617552. URL: https://doi.org/10.1145/2617548.2617552.

[34] Google Security Blog. *Rust in Android: Move Fast and Fix Things*. Google Online Security Blog. 2025. URL: https://security.googleblog.com/2025/11/rust-in-android-move-fast-fix-things.html.

[35] The Linux Foundation. *Rust Support in the Linux Kernel*. Linux Kernel Documentation. 2026. URL: https://docs.kernel.org/rust/index.html.